Retail Inventory Management System

TEAM MEMBERS:

- PANJA SHARATH CHANDRA AM.SC.U4CSE24138
- RACHARLA SRI CHARAN AM.SC.U4CSE24147
- REVANNAGARI NISHANTH AM.SC.U4CSE24150
- KANDURI VENKATA SAI MANI AVINASH AM.SC.U4CSE24171

AIM OF THE PROJECT:

The main aim of the **Retail Inventory Management System (RIMS)** is to design and implement a **centralized database-driven system** that automates the process of managing products, suppliers, sales, and stock levels in a retail store. The project seeks to enhance efficiency, accuracy, and decision-making by eliminating manual record-keeping and providing real-time insights into inventory movement, sales trends, and supplier performance.

Through the use of relational database concepts, normalization, and SQL-based operations, the system ensures **data integrity, reduced redundancy, and easy scalability** for integration with advanced modules like billing, POS systems, and analytics.

FRONTEND AND BACKEND IMPLIMENTATION:

The backend is implemented using **PostgreSQL**, and SQL queries handle the CRUD operations (**Create, Read, Update, Delete**) efficiently. It can be integrated with a frontend application built using **Java, PHP, or Python** for user interaction. The database includes tables for **Products, Suppliers, Customers, Sales, Purchases, and Inventory**, all interconnected through primary and foreign key relationships.

Additionally, the system supports **report generation** for:

- Daily and monthly sales summaries
- Low-stock alerts
- Supplier order reports
- Revenue and profit tracking

This project demonstrates practical DBMS concepts such as **data modeling, normalization, relational schema design, query optimization, and report generation**. It not only improves business efficiency but also provides valuable analytical insights for retail management.

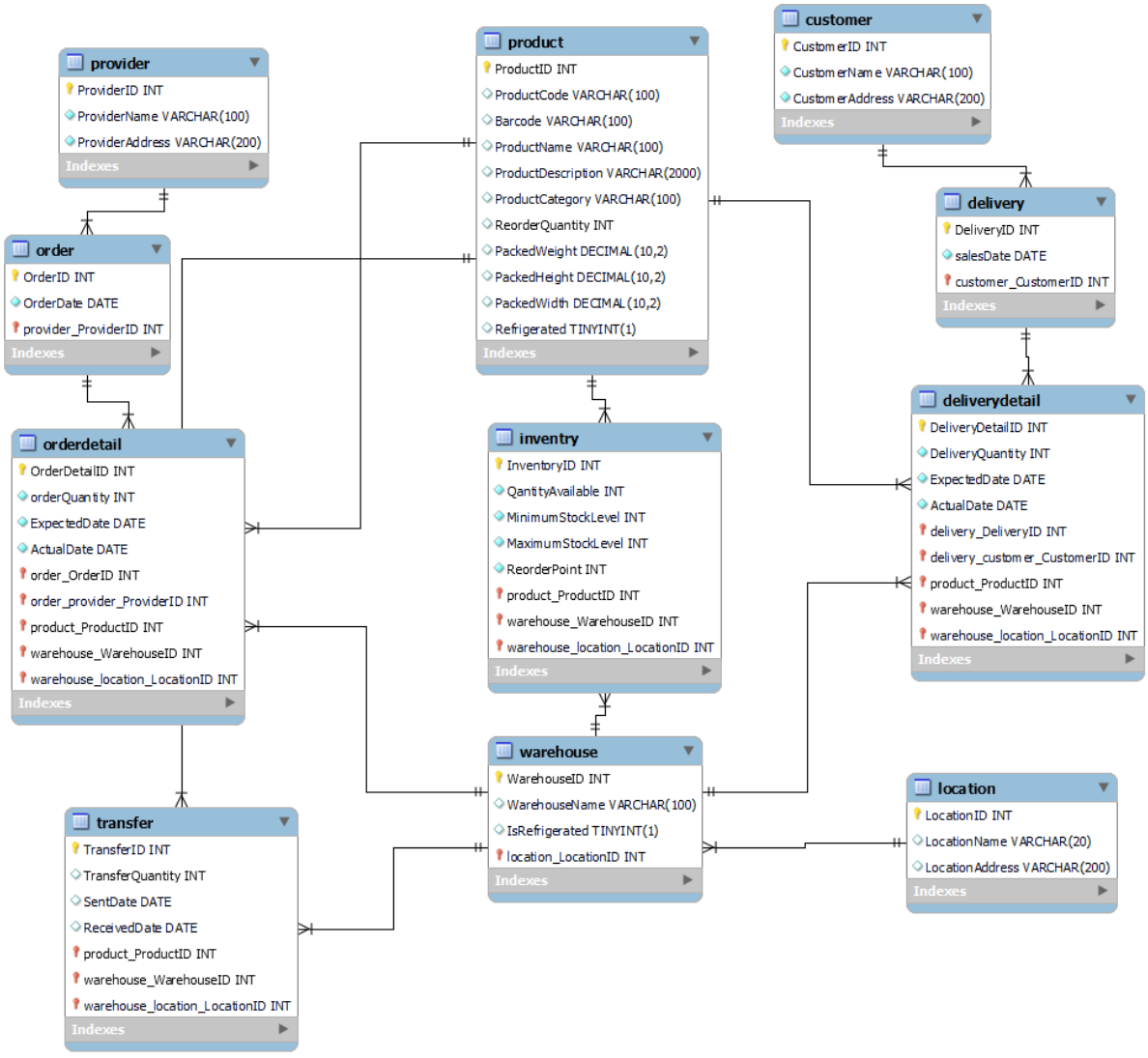
Objectives

1. To design a relational database system for managing retail inventory.
2. To automate product tracking, stock updates, and transaction records.
3. To maintain data integrity and minimize redundancy through normalization.
4. To generate analytical reports for sales and stock performance.
5. To support easy retrieval and updating of inventory data through SQL operations.

Key Features

1. **Product Management** – Add, update, and delete product details with category, price, and stock information.
2. **Supplier Management** – Maintain supplier records with contact details and product supply history.
3. **Purchase & Order Management** – Record incoming stock from suppliers and automatically update inventory levels.
4. **Sales & Delivery Management** – Manage customer orders, generate bills, and update stock in real-time.
5. **Inventory Tracking** – Monitor available stock, set reorder points, and trigger low-stock alerts.
6. **Warehouse Management** – Track stock across multiple warehouse locations with refrigeration and capacity details.
7. **Transfer Module** – Record internal stock transfers between warehouses and maintain consistency in inventory data.
8. **Report Generation** – Generate daily and monthly sales summaries, low-stock alerts, supplier order reports, and profit analysis.
9. **Database Integrity & Normalization** – Ensure consistency and eliminate redundancy through a properly normalized relational schema.
10. **Scalability & Integration** – Can be easily integrated with frontend systems or extended to include billing, staff, and analytics modules.

Database Design (ER Diagram)



Entities and Attributes

- **Provider**(**ProviderID**, ProviderName, ProviderAddress)
- **Product**(**ProductID**, ProductCode, Barcode, ProductName, ProductDescription, ProductCategory, ReorderQuantity, PackedWeight, PackedHeight, PackedWidth, Refrigerated)
- **Customer**(**CustomerID**, CustomerName, CustomerAddress)
- **Order**(**OrderID**, OrderDate, ReorderDate, provider_ProviderID)
- **OrderDetail**(**OrderDetailID**, OrderQuantity, ExpectedDate, ActualDate, order_OrderID, product_ProductID, order_provider_ProviderID, warehouse_WarehouseID, warehouse_location_LocationID)

- **Inventory**(**InventoryID**, QuantityAvailable, MinimumStockLevel, MaximumStockLevel, ReorderPoint, product_ProductID, warehouse_WarehouseID, warehouse_location_LocationID)
- **Warehouse**(**WarehouseID**, WarehouseName, IsRefrigerated, location_LocationID)
- **Location**(**LocationID**, LocationName, LocationAddress)
- **Delivery**(**DeliveryID**, SalesDate, customer_CustomerID)
- **DeliveryDetail**(**DeliveryDetailID**, DeliveryQuantity, ExpectedDate, ActualDate, delivery_DeliveryID, delivery_customer_CustomerID, product_ProductID, warehouse_WarehouseID, warehouse_location_LocationID)
- **Transfer**(**TransferID**, TransferQuantity, SentDate, ReceivedDate, product_ProductID, warehouse_WarehouseID, warehouse_location_LocationID)

Relationships

- **Provider → Order** (One-to-Many)
- **Order → OrderDetail** (One-to-Many)
- **Product → OrderDetail** (One-to-Many)
- **Product → Inventory** (One-to-Many)
- **Warehouse → Inventory** (One-to-Many)
- **Location → Warehouse** (One-to-Many)
- **Warehouse → Transfer** (One-to-Many)
- **Product → Transfer** (One-to-Many)
- **Customer → Delivery** (One-to-Many)
- **Delivery → DeliveryDetail** (One-to-Many)
- **Product → DeliveryDetail** (One-to-Many)
- **Warehouse → DeliveryDetail** (One-to-Many)

Expected Outcomes

- Automated and reliable inventory management
- Reduced human errors and time wastage
- Easy report generation for decision-making
- Centralized, secure, and scalable data storage

Conclusion:

The **Retail Inventory Management System** provides a **structured and automated solution** for managing inventory in a retail environment. By applying database principles like normalization, relational schema design, and SQL querying, the project ensures efficient data handling and insightful analytics. It enhances operational efficiency, accuracy, and business decision-making for retailers.